

# Advanced Sorting

Comparable vs. Comparator Interfaces

**CODEHIVE**

## 1. Beyond Primitive Sorting

While `Collections.sort()` works instantly on Strings and Integers, custom objects (like Car or User) require a sorting "rule." Java provides two ways to define these rules: **Comparable** and **Comparator**.

### The Core Difference:

- **Comparable** is for "Natural Sorting" (default).
- **Comparator** is for "Custom Sorting" (extra options).

## 2. The Comparable Interface

Implement Comparable in the class itself to define its default order. It uses the `compareTo()` method.

```
class Car implements Comparable<Car> {  
    int year;  
    // ... constructor ...  
  
    public int compareTo(Car other) {  
        return this.year - other.year;  
    }  
}
```

Usage: `Collections.sort(carList);` (No extra arguments needed).

### 3. The Comparator Interface

When you cannot modify a class, or when you need multiple sorting ways (by Name, by Rank, by Age), use `Comparator`. It uses the `compare()` method.

```
class SortByYear implements Comparator<Car> {  
    public int compare(Car a, Car b) {  
        return a.year - b.year;  
    }  
}
```

Usage: `Collections.sort(carList, new SortByYear());`

## 4. Implementation: Car Sorting

Let's look at a full example of sorting a list of Cars by their manufacturing year using a separate Comparator class.

```
ArrayList<Car> myCars = new ArrayList<>();  
myCars.add(new Car("BMW", "X5", 1999));  
myCars.add(new Car("Ford", "Mustang", 1970));  
  
Collections.sort(myCars, new SortByYear());
```

## 5. Modern Lambdas as Comparators

With Java 8+, you don't need to create a whole new class for simple sorting. You can use a Lambda expression directly inside the sort method.

```
// Sorting by year using Lambda
myCars.sort((a, b) -> a.year - b.year);

// Sorting by Brand
myCars.sort((a, b) -> a.brand.compareTo(b.brand));
```

## 6. Sorting Logic (The Int Return)

Both interfaces rely on an integer return value to determine order:

- **Negative (-1):** "This" comes before "That".
- **Zero (0):** Both are equal.
- **Positive (1):** "This" comes after "That".

Pro Tip: For integers, `a - b` is a shorthand that works perfectly for these rules!

## 7. Multi-Criteria Sorting

What if two cars have the same year? You can sort by year, and then by brand name.

```
myCars.sort(Comparator.comparing(Car::getYear)
                .thenComparing(Car::getBrand));
```



## 8. Comparison Matrix

| Feature         | Comparable                  | Comparator                       |
|-----------------|-----------------------------|----------------------------------|
| Modification    | Modified original class     | External to original class       |
| Package         | java.lang                   | java.util                        |
| Number of rules | One (Natural Order)         | Unlimited multiple rules         |
| Method          | <code>compareTo(obj)</code> | <code>compare(obj1, obj2)</code> |

## 9. Summary & Final Tips

- Use **Comparable** if there is one obvious way to sort the data.
- Use **Comparator** if you want different sorting strategies.
- Use **Lambdas** or **Method References** for modern, readable sorting.
- Remember: for descending order, just swap the parameters:  $(b, a) - > a - b$  style logic.

© 2026 CodeHive Academy | Advanced Java Series